

L14: GNN Training and Decoders

Let us now see how to train a GNN, i.e., how to compute the model parameters given some training data.

For simplicity, let us consider a binary node classification task in which we aim to distinguish whether each node of an online social network is human (class-0) versus bot (class-1). Classification tasks with a larger number of classes, or non-classification tasks (such as node clustering, link prediction, etc) can be performed similarly, with minor modifications in the following loss function.

Suppose that the neural network consists of K hidden layers. The final embedding of each node $\mathbf{v}$ is denoted by $\mathbf{z}_v = \mathbf{h}_v^K$.

We need to also design the "decoder" part of the neural network, which maps the embedding vector $\mathbf{z}_v$ to the corresponding class of node $\mathbf{v}$. The simplest decoder is a "**softmax**" operator with parameters $\mathbf{D}$, where $\mathbf{D}$ is a vector of the same dimensionality as the embedding vector. The output of the softmax is given by the activation function of the dot-product between the transpose of the embedding vector $\mathbf{z}_v$ and $\mathbf{D}$:

$$\sigma(\mathbf{z}_v^T \mathbf{D})$$

So, if the softmax output for node $\mathbf{v}$ is closer to 0, we predict that node $\mathbf{v}$ is in class-0, and if it is closer to 1 we predict that node $\mathbf{v}$ is in class-1.

How can we compute all the parameters of the GNN model, including both the encoder (the part of the network that computes the embeddings with parameters $\mathbf{W}_k$ and $\mathbf{B}_k$ for $k=1,2,\dots,K$) and the decoder (with parameters $\mathbf{D}$)?

Suppose that we are given some training data: the actual class y_v (0 or 1) for a set $\mathbf{V}'$ of nodes in the network. We can then compute the following loss function, which is known as *cross-entropy loss*:

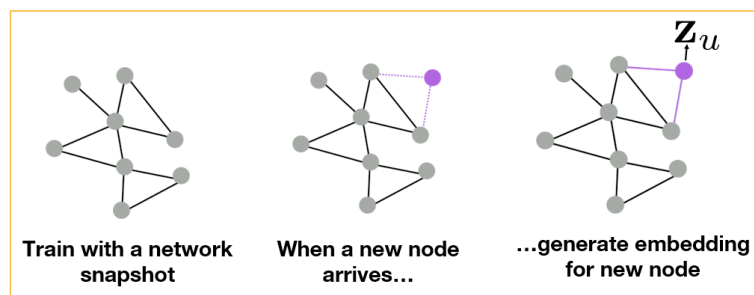
$$\mathcal{L} = - \sum_{v \in \mathbf{V}'} [y_v \log(\sigma(\mathbf{z}_v^T \mathbf{D})) + (1 - y_v) \log(1 - \sigma(\mathbf{z}_v^T \mathbf{D}))]$$

Note that when the softmax output for node $\mathbf{v}$ is close to the actual class of that node, i.e., when $y_v \approx \sigma(\mathbf{z}_v^T \mathbf{D})$, the contribution of node $\mathbf{v}$ in the loss is almost 0. Otherwise, when the model predicts the wrong class for node $\mathbf{v}$, i.e., when $y_v + \sigma(\mathbf{z}_v^T \mathbf{D}) \approx 1$, node $\mathbf{v}$ contributes significantly to the loss.

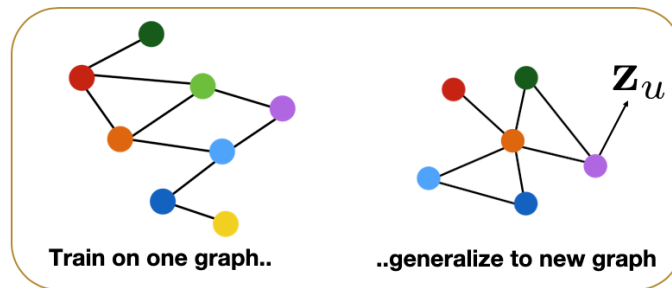
So, we can train the neural network to minimize the previous loss function by selecting appropriate model parameters: $\mathbf{W}_k$, $\mathbf{B}_k$ ($k=1,2,\dots,K$) and $\mathbf{D}$. This optimization is not convex, and so all neural network learning algorithms rely on numerical approaches such as variations of *Stochastic Gradient Descent* (SGD).

After we have computed the previous parameters, we can use the GNN model to classify any node in the network, including those for which we do not have training data.

Additionally however, we can compute embeddings (and then classify) even for nodes that were not present in the original network. The following figure illustrates this application of GNNs in the context of dynamic graphs. Suppose that the GNN was trained based on the grey portion of the graph. At some point later, node u joins the network. We can still use the same GNN, without retraining it, to compute the embedding of node u -- and then to classify it. Obviously this approach will only work as long as the new nodes that join the network follow the same structural properties that the original network had.



Another application of GNNs is to generalize from one or more given graphs to entirely new graphs -- as long as we have reasons to believe that the new graphs have the same structural properties with the given graphs. For instance, in the context of protein-protein interaction networks, we may have such graphs from closely related bacterial species (e.g., *Salmonella typhi* and *Salmonella typhimurium*). We can then learn a GNN model based on these known graphs and apply that GNN to predict properties of proteins in novel *Salmonella* species, as shown in the following Figure.



Food For Thought

- How would you modify the previous loss function in the case of multiple classes (multinomial classification)?
- Suppose that the task is not how to classify nodes -- but to predict the presence of links between nodes. How would you train the GNN model in that case given a training graph?